

LUMEN: Coordinated Failure Recovery for Distributed LLM Serving

Zhang Cao¹, Shujie Han², Juncheng Zhang¹, Yuanming Ren¹, Yongkun Li³, Patrick P. C. Lee¹

¹The Chinese University of Hong Kong ²Northwestern Polytechnical University

³University of Science and Technology of China

Abstract

Modern large language model (LLM) serving clusters distribute inference requests across multiple worker processes on different GPUs, but failures are prevalent at scale. When a worker fails, the cluster simultaneously loses the failed worker’s GPU-resident key-value (KV) caches and serving capacity, leaving surviving workers to absorb the redirected traffic while re-running interrupted requests from scratch. Existing fault-tolerant systems either restart interrupted requests from scratch or restore KV caches from checkpoints stored on a fixed neighboring worker, but both approaches route recovery work without considering current cluster load and leave the recovering worker idle during model reload. We present LUMEN, a fault-tolerant LLM serving system that treats recovery as a load-aware coordination problem across three decision points: checkpoint placement before failures, interrupted-request distribution at failure time, and serving capacity restoration during model reload. We evaluate LUMEN using both prototype experiments and large-scale simulations and demonstrate significant improvements in serving and recovery times.

1 Introduction

Large language models (LLMs) are widely deployed as latency-sensitive online services, including conversational assistants, code completion, and real-time content generation, that must sustain interactive response times under continuous load [20, 42, 43]. Since frontier models [11, 35] exceed the memory of a single GPU, production deployments shard model copies using tensor or pipeline parallelism across large-scale GPU clusters spanning more than 10,000 GPUs to scale serving capacity [18, 19, 45], with each model copy independently served by a process called *worker*. Unfortunately, failures are prevalent in large GPU clusters at deployment scale. Production traces report multiple hardware incidents per day, and LLM serving jobs encounter failures every few hours on average [15, 18, 19, 45]. These failures originate at the GPU, node, or inter-node network layer, but they all manifest at the serving layer as one or more workers that become unavailable.

Existing failure recovery strategies in LLM serving fall into either *Stop-and-Restart* or *Fixed-Checkpointing*. Stop-and-Restart, the default in modern serving systems [1, 4–6], reloads the failed worker and dispatches its interrupted requests to surviving workers, where each re-runs requests

from scratch to rebuild the lost KV cache. This requires no checkpointed state, but imposes expensive replay costs on long-context requests and overwhelms already-stressed surviving workers. Our analysis (§3) shows that in recovering a single-worker failure, Stop-and-Restart increases the mean time-to-first-token (TTFT) by $4.0\times$ and the mean time-per-output-token (TPOT) by $1.6\times$ across the surviving cluster, and the increases persist at scale from 4 to 64 workers. Fixed-Checkpointing, exemplified by DéjàVu [32], avoids full replays by streaming each request’s KV cache to a pre-determined worker as a checkpoint holder before any failure occurs; after a failure, all interrupted requests are restored from that worker. However, the checkpoint holder is chosen statically rather than by current load, so recovery work could be concentrated on a heavily loaded worker regardless of its queue depth. Both approaches also leave the recovering worker idle until the full model is reloaded. These limitations motivate *load-aware* recovery that uses current load status across workers to guide three recovery decisions: (i) where to place KV checkpoints before failures, (ii) where to route interrupted requests at failure time, and (iii) how to utilize the recovering worker during model reload.

We present LUMEN, a fault-tolerant, load-aware recovery system for LLM serving. Our key insight is that fast recovery from worker-level failures can be formulated as a load-aware coordination problem. LUMEN comprises three coordinated mechanisms: (i) *load-aware KV checkpointing*, which places each request’s KV checkpoint on the worker expected to have the least recovery load, spreading checkpoints across the cluster rather than concentrating them on one fixed neighbor; (ii) *locality-aware recovery scheduling*, which routes each interrupted request to its checkpoint holder, redirecting only requests with small checkpointed prefixes when a holder would still be overloaded; and (iii) *speculation-assisted progressive recovery*, which extends speculative decoding [12, 21] for recovery by loading a lightweight draft model on the recovering worker while the full model is reloaded in the background, thereby enabling the recovering worker to immediately contribute temporary capacity through speculative decoding without staying idle.

We implement and evaluate LUMEN as a prototype built on SGLang [43] and as a large-scale simulator built on Vidur [9]. On a four-worker prototype serving Qwen3-32B [41], LUMEN reduces mean TTFT by 44.4% and 7.1%, mean TPOT by 15.9% and 7.0%, and recovery time by 50.0% and 34.9% over Stop-and-Restart and Fixed-Checkpointing, respectively;

on an eight-worker prototype serving Qwen3-14B [41], LUMEN reduces mean TTFT by 29.6% and 15.9%, mean TPOT by 7.1% and 4.2%, and recovery time by 64.1% and 63.9% over the same baselines. Large-scale simulations confirm LUMEN’s performance gains for cluster sizes of up to 64 workers. We will open-source LUMEN’s prototype and simulator in the final paper.

2 Background

2.1 LLM Serving Basics

Prefill, decode, and KV cache. LLM inference follows an autoregressive Transformer model [36]. A request first runs a *prefill*, which processes all prompt tokens in parallel and generates the first output token. To bound per-step compute and interleave prefill with ongoing decodes, modern LLM serving systems split long prompts into fixed-size chunks (e.g., 1,024 tokens per chunk) and process them across iterations [10]. The model then enters the *decode* phase, which generates one token per step, conditioned on the prompt and all previously generated tokens [26, 44]. To avoid recomputing attention over the full prefix at every decode step, the system caches per-token keys and values as the *KV cache*. Following paged KV management [20], we refer to a fixed-size block of cached keys and values for contiguous token positions as a *KV page*. The KV cache grows with sequence length and consumes a large fraction of GPU memory [23, 29]. Resuming a request after a failure without re-running prefill requires preserving these KV pages.

Speculative decoding. As the decode phase is memory-bandwidth-bound, GPU compute units often sit idle while waiting for model weights and KV cache. *Speculative decoding* [12, 21] turns linear decoding into parallel verification: a small *draft model* rapidly proposes a sequence of draft tokens, and the large *target model* verifies them in a single forward pass, effectively performing a mini-prefill step. Token acceptance is sequential: all draft tokens up to the first rejected position are accepted, and the target model contributes one additional corrected token at the rejection point; remaining draft tokens are discarded. As the target model verifies multiple tokens in parallel, it increases throughput without changing the output distribution [21]. Efficiency depends on the draft model’s acceptance rate and its per-step overhead relative to the target-model decode step.

Parallelism. Production deployments of large models shard one model copy across multiple GPUs using two complementary schemes: *tensor parallelism (TP)* shards each layer across GPUs within a node and issues fine-grained collectives every decode step; *pipeline parallelism (PP)* partitions layers into stages placed on different GPUs or nodes and exchanges only activation tensors between stages, at the cost of pipeline bubbles and more complex cross-stage state management. A *worker* is a complete copy of the model weights

that can independently serve requests; in practice, a worker may occupy one GPU, a TP group within one node, a PP group across stages, or a combined TP+PP group. The GPUs inside a worker are tightly coupled: the failure of any one GPU disables the entire worker.

2.2 Failures in Distributed LLM Serving

Failure model. Production-serving deployments share the same GPU hardware and interconnect stack as training clusters and are hence exposed to the same hardware and network failures. These failures originate at three physical layers. (i) *GPU-level failures* come from ECC errors, overheating shutdown, PCIe/NVLink errors, driver resets, CUDA runtime aborts, or out-of-memory kills; losing any worker process halts the entire TP or PP group. (ii) *Node-level failures* come from a host crash, kernel panic, power event, or planned reboot; they disable all GPUs and serving processes on the node simultaneously, taking down all co-located workers at once. (iii) *Inter-node network failures* come from NIC failures, link flaps, switch port errors, fabric partitions, or persistent stragglers; they interrupt cross-worker activation exchanges, forcing the affected worker to be declared unavailable once collective latency violates the service-level objective (SLO).

All three failure types cause one or more workers to become unavailable, which we call a *worker-level failure*. The unavailable worker is the *failed worker*, which loses its GPU-resident KV cache, while the cluster loses that worker’s prefill and decode capacity until model reload completes. In this work, we model each failure conservatively as requiring a full model reload, which is the dominant cost for node-level and persistent GPU-level failures, and address worker-level failures regardless of their underlying cause.

Recovery flow. When a worker-level failure occurs, the workers that remain available are the *surviving workers*; the failed worker after restart, while reloading its model, is the *recovering worker*. We focus on two recovery baselines (§1): *Stop-and-Restart*, which re-runs prefill from scratch for every interrupted request on surviving workers, and *Fixed-Checkpointing*, which restores interrupted requests from KV checkpoints stored on a fixed neighboring worker, which we call the request’s *checkpoint holder*. Both baselines leave the recovering worker idle until its model finishes reloading.

3 Motivation

We characterize the recovery cost of Stop-and-Restart under worker failures to identify the root causes that LUMEN targets. We select Stop-and-Restart as the primary baseline as it represents the default deployment [1, 4–6]. Also, it isolates the costs of lost KV cache and serving capacity without introducing the load imbalance associated with Fixed-Checkpointing.

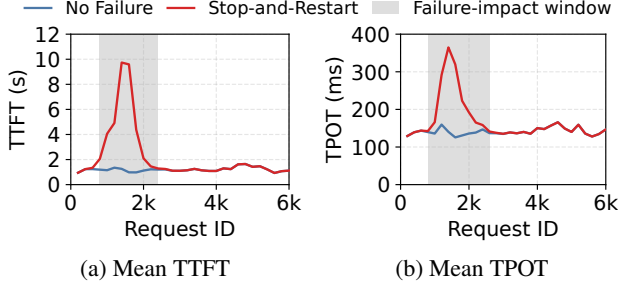


Figure 1: Single-worker failure in a simulated four-worker cluster.

3.1 Impact of Worker Failures

Observation 1: A single-worker failure degrades both TTFT and TPOT cluster-wide. We use our simulator to run Llama-3-70B [24] on a cluster of four workers driven by the Splitwise-Conv trace [26]. We evaluate Stop-and-Restart by failing a single worker after the cluster reaches steady state and compare it with the No-Failure case. Figure 1 shows mean TTFT and TPOT in buckets across request IDs (with 200 requests per bucket), averaged over five runs, and marks the *failure-impact window* (i.e., the period from when the failure occurs until the cluster recovers to normal performance); see the failure-impact window definition and detailed cluster setup in §6.1. Within the failure-impact window, the mean TTFT increases from 1.16 s to 4.69 s (4.0 $\times$) and the mean TPOT increases from 138.9 ms to 224.6 ms (1.6 $\times$). Per-request traces show that only 2.7% of requests in this window are in flight on the failed worker; the remaining 97.3% run on surviving workers. Thus, the failure degrades latency cluster-wide, affecting not only the failed worker’s own requests but also requests on surviving workers. Once the recovering worker rejoins, both metrics return to their No-Failure levels.

Observation 2: Failure-induced degradation persists at larger cluster sizes. We scale the simulated cluster from 4 to 64 workers, maintaining a fixed load of 1.4 QPS per worker to evaluate scalability. After the cluster reaches steady state, we fail 25% of the workers (1 to 16 failed workers). Figure 2 shows the mean TTFT and mean TPOT over the failure-impact window for No-Failure and Stop-and-Restart. Stop-and-Restart degrades latency by a similar factor at every cluster size: the mean TTFT stays in 4.1–4.7 s, about 4 $\times$ the No-Failure case of around 1 s, and the mean TPOT stays near 230 ms, about 1.6 $\times$ No-Failure of around 140 ms.

3.2 Root Cause Analysis

We analyze the root causes of Observations 1 and 2 by categorizing requests in the failure-impact window. For *interrupted requests* (i.e., requests running on a failed worker), Stop-and-Restart re-prefills them from scratch using each request’s retained token history to rebuild the lost KV cache. For *uninterrupted requests* (i.e., remaining requests, including those already running on surviving workers and newly arrived re-

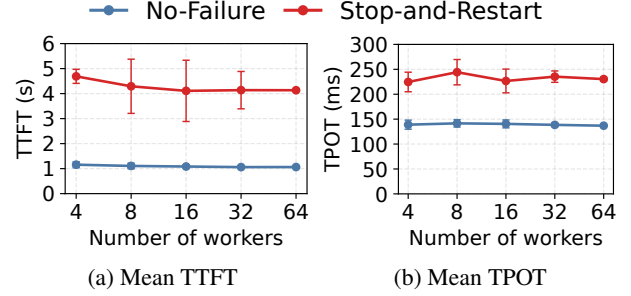


Figure 2: Worker failures versus cluster size.

Table 1: Breakdown of Stop-and-Restart. Values are mean $\pm$ 95% confidence intervals over five runs.

Workers	Uninterrupted TTFT (s)		Interrupted Replay TTFT (s)
	Total	Queueing	
4	4.1 $\pm$ 0.2	3.3 $\pm$ 0.2	25.6 $\pm$ 7.1
8	3.8 $\pm$ 1.1	3.0 $\pm$ 1.0	24.0 $\pm$ 1.4
16	3.5 $\pm$ 1.2	2.8 $\pm$ 1.2	27.1 $\pm$ 1.7
32	3.5 $\pm$ 0.7	2.7 $\pm$ 0.6	29.4 $\pm$ 2.9
64	3.6 $\pm$ 0.1	2.8 $\pm$ 0.1	27.8 $\pm$ 0.7

quests during model reload), they do not lose KV cache on failure but may queue behind redirected traffic. Within each uninterrupted request’s TTFT, we measure the queueing delay before prefill starts (the residual time is the prefill execution time). Table 1 shows the breakdown.

Observation 3: Model reloading delays uninterrupted requests due to capacity loss. While a failed worker reloads its model, its incoming traffic is redirected to surviving workers. At a fixed 25% failure rate, this increases the load on every surviving worker by one-third regardless of cluster size (the lost 25% of load is absorbed by the remaining 75% of workers, i.e., a one-third increase). For TTFT, uninterrupted requests do not lose KV state but still wait behind redirected traffic. Table 1 shows that their mean TTFT stays within 3.5–4.1 s at every cluster size, dominated by queueing delay (78–80%). For TPOT, the same capacity shortfall increases decode contention on surviving workers, matching the roughly 1.6 $\times$ TPOT degradation in Figure 2(b).

Observation 4: Lost KV cache forces expensive replay for interrupted requests. We measure *replay TTFT* for each interrupted request as the time from its original arrival to the first output token of the re-prefill attempt; it counts (i) the work already spent before the failure, (ii) any queueing after dispatch, and (iii) the full re-prefill required to rebuild the KV cache. The replay TTFT is within 24.0–29.4 s across cluster sizes, roughly 5.9–8.4 $\times$ the uninterrupted requests’ TTFT. The per-request re-prefill cost is similar across cluster sizes, with only slight variation due to queueing behind redirected traffic on overloaded surviving workers. Although interrupted requests are only 2.7% of the failure-impact window (Observation 1), their high replay TTFT increases the overall window

mean to 4.69 s.

Recovery objective. We aim to minimize recovery costs cluster-wide. This requires load-aware recovery decisions that (i) preserve KV checkpoints to eliminate replay TTFT for interrupted requests, and (ii) recover capacity during model reload to mitigate queueing delay (TTFT) and decode contention (TPOT) for uninterrupted requests.

3.3 Challenges

Our recovery optimization faces three design challenges.

Challenge 1: Checkpoint placement. Storing in-flight KV caches on other workers bypasses expensive re-prefills. However, Fixed-Checkpointing (§1) [32] statically maps all requests from a failed worker to a single neighbor (i.e., the checkpoint holder). This creates severe recovery bottlenecks where queueing negates KV-reuse benefits. Alternatively, remote storage offloading [29] avoids host memory pressure but suffers from unpredictable network latency during recovery. The challenge is dynamically distributing checkpoints to prevent recovery hotspots, while bounding host memory usage and keeping restore costs lower than recomputation in practice.

Challenge 2: Recovery dispatch. At failure time, routing interrupted requests presents a trade-off between KV reuse and load balancing. Maximizing KV reuse by sending all requests to their checkpoint holders creates severe recovery hotspots. Conversely, rebalancing requests to underloaded workers discards checkpointed KV pages, forcing expensive full re-prefills. The challenge is to dynamically strike a balance by redirecting only requests with minimal checkpointed prefixes away from overloaded holders, ensuring load balance while bounding recomputation penalties.

Challenge 3: Reload-window assistance. Recovering workers traditionally remain idle during lengthy model reloads, while surviving workers are bottlenecked by the decode throughput of redirected traffic [26]. To extract usable decode throughput immediately, we repurpose speculative decoding [21, 22, 25] for fault tolerance: the recovering worker rapidly loads a lightweight draft model to produce tokens for an overloaded surviving worker to verify. While this advances multiple tokens per step to swiftly clear backlogs, it introduces two critical difficulties. First, verifying stale drafts on an already-stressed worker degrades performance; this overhead must therefore be bounded to drop stale tokens without stalling the normal decode path. Second, the recovering worker must seamlessly transition from draft assistance back to full service without incurring another disk-loading stall at the end of the reload window.

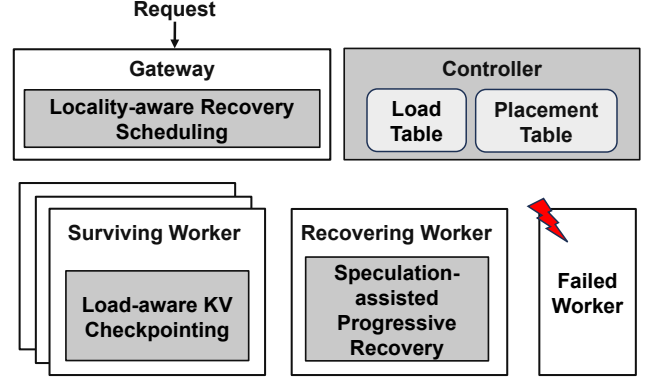


Figure 3: LUMEN overview.

4 LUMEN Design

LUMEN addresses the challenges in §3.3 via three coordinated mechanisms: *load-aware KV checkpointing*, *locality-aware recovery scheduling*, and *speculation-assisted progressive recovery*. All three mechanisms share a common principle that recovery decisions follow current load observations and direct extra work toward less-loaded workers.

4.1 Design Overview

Architecture. Figure 3 shows the LUMEN architecture. In existing LLM serving systems (e.g., SGLang [43]), a *gateway* merely routes requests and triggers naive Stop-and-Restart recovery upon failure, while workers independently manage local GPU memory. LUMEN augments the gateway to retain the *token history* of every in-flight request, including the prompt and accumulated output token IDs. It also introduces a centralized *controller*, which maintains a *load table* of per-worker queueing and capacity signals and a *placement table* that maps each request to its checkpoint holder; both tables drive checkpoint placement in steady state and request dispatch when a failure occurs.

Workflow. In steady state, load-aware KV checkpointing (§4.2) continuously copies each request’s new KV pages to its checkpoint holder’s CPU memory off the GPU critical path; the controller reads the load table to choose that checkpoint holder and records the choice in the placement table. When a worker fails, the controller triggers two parallel actions: locality-aware recovery scheduling (§4.3), which dispatches each interrupted request to its checkpoint holder for KV reuse and redirects some requests away from overloaded checkpoint holders to avoid hotspots, and speculation-assisted progressive recovery (§4.4), which keeps the recovering worker useful while it reloads the target model.

Goals. LUMEN targets three design goals.

- **Fast recovery**, which lowers post-failure latency by jointly preserving in-flight KV cache and recovering lost serving capacity before target-model reload completes.

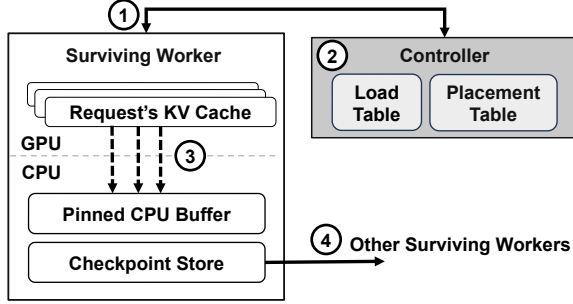


Figure 4: Load-aware KV checkpointing.

- **Low steady-state overhead**, which keeps the always-on checkpoint path off the GPU decode path and bounded in host memory so that failure-free TTFT and TPOT match those of a serving system without LUMEN.
- **Scalability**, which preserves the recovery benefit across cluster sizes and across concurrent multi-worker failures.

4.2 Load-aware KV Checkpointing

To address Challenge 1, LUMEN introduces load-aware KV checkpointing. Rather than routing all checkpoints to a static worker [32], it evaluates real-time load signals to distribute checkpoints across multiple workers at request granularity.

Checkpoint placement. Figure 4 shows LUMEN’s KV checkpoint placement. The controller aggregates per-worker feedback (①) into a load table, which then drives the placement table (②) to map each active request to its designated checkpoint holder. To estimate potential restore pressure, the load table tracks three per-worker states: available checkpoint-store capacity, queueing delay, and reserved KV checkpoint footprints. Rather than relying on periodic polling, LUMEN employs event-driven updates: it adjusts reserved KV checkpoint footprints instantly upon allocation or release, and updates queue delays as workers complete requests.

Load-table updates impose minimal overhead by decoupling the controller from heavy data transfers. The controller operates at request granularity rather than token granularity and exchanges only lightweight metadata (e.g., checkpoint-holder IDs, request IDs, capacities, and queueing delays) during prefill and request completion. Consequently, control traffic scales with request throughput, independent of token generation or sequence length. Meanwhile, large KV pages stream directly peer-to-peer between workers, completely bypassing the controller.

LUMEN designates a single worker to store the KV checkpoint for each active request. It requires that the designated worker’s available host memory can accommodate the request’s *reserved KV checkpoint footprint*, a conservative allocation derived from the maximum context length rather than the current dynamic KV size. By reserving this footprint upon assignment and releasing it upon completion, LUMEN prevents checkpoint holders from exhausting host memory. Also,

maintaining exactly one checkpoint per request bounds steady-state memory usage and simplifies placement logic. LUMEN bounds checkpoint memory usage with a per-worker host-memory limit, using the same budget as Fixed-Checkpointing (e.g., DéjàVu [32]); in our prototype deployments (§6.1), this limit is 80 GB per worker for Qwen3-14B and 160 GB per worker for Qwen3-32B [41]. Fallback mechanisms for checkpoint-holder failures are detailed in §4.5.

LUMEN scores each candidate worker by two metrics: (i) *queueing delay* q_w measures the average wait time on worker w from request arrival to prefill initiation; and (ii) *restore pressure* $p_w(r)$ estimates the expected mean KV restore latency for worker w if request r is assigned to it. To compute $p_w(r)$, the controller first averages the reserved KV footprints of the checkpoints that w would hold after assigning r , then divides this average by a fixed host-to-GPU bandwidth [14]. These metrics capture distinct latency dimensions: q_w reflects current congestion, while $p_w(r)$ infers the future recovery load concentrated on w during a failure. LUMEN balances them using a tunable weight λ to accommodate variations in request rate, model size, and hardware bandwidth.

LUMEN assigns the KV checkpoint to the candidate worker that minimizes the weighted sum of these metrics. The candidate set $F(r)$ comprises all workers with sufficient storage to accommodate r ’s reserved KV footprint, excluding the worker currently serving r . This physical separation ensures that no single-worker failure can compromise both a request’s active KV cache and its checkpoint. The resulting checkpoint holder, $h(r)$, is:

$$h(r) = \arg \min_{w \in F(r)} (q_w + \lambda p_w(r)). \quad (1)$$

We set $\lambda = 1$ by default and evaluate its sensitivity (§6). LUMEN finalizes the checkpoint holder $h(r)$ upon prefill completion, logging this assignment in the placement table for the controller to route interrupted requests during recovery. Subsequently, all KV pages are continuously streamed to this designated checkpoint holder until the request terminates and its checkpoint is released. Notably, this placement strategy is intentionally asymmetric: it proactively shifts future recovery load toward workers with greater spare capacity by allocating them more KV checkpoints.

Incremental checkpoint pipeline. Figure 4 also shows that LUMEN asynchronously offloads KV cache to checkpoint holders without blocking the GPU critical path. Each worker provisions two host-memory regions: a *pinned CPU buffer* for staging local GPU-to-network transfers, and a *checkpoint store* for persisting incoming remote KV pages for checkpointing. To minimize overhead, workers track checkpoint progress and transmit only newly generated KV pages after each prefill or decode batch. A background thread copies these pages from GPU memory to the local pinned buffer (③), then streams them over the network to the designated checkpoint store (④). This two-stage pipeline completely overlaps with normal inference execution.

To facilitate recovery, each transferred page carries a *page tag*: a hash value of the token IDs stored in page i , paired with the page’s end position in the request token sequence (i.e., the cumulative token length after page i). Since this tag derives purely from the request’s token sequence, any worker can independently reconstruct it using the token history retained by the gateway. To guarantee atomicity, pages become visible in the checkpoint store only upon complete reception; if a failure interrupts a transfer, the lookup simply halts at the last valid tag, leaving only the suffix after the last complete page for recomputation. Consequently, these page tags enable a recovering worker to pinpoint the longest contiguous checkpointed prefix (§4.3).

4.3 Locality-aware Recovery Scheduling

LUMEN addresses Challenge 2 via locality-aware recovery scheduling. Upon failure detection by the gateway, the controller initially routes interrupted requests to their checkpoint holders to maximize KV reuse. To prevent these holders from becoming recovery hotspots, it dynamically sheds load by redirecting a subset of requests to less-loaded workers, deliberately trading KV reuse for recomputation.

Recovery dispatch. To prioritize KV reuse, the controller routes interrupted requests to their designated checkpoint holders. If a checkpoint holder has also failed, LUMEN marks the request for recomputation and assigns it to the least-loaded surviving worker. While this locality-first approach preserves cache state, relying on historical load observations risks concentrating recovery traffic on specific workers, necessitating the subsequent rebalancing step.

Load rebalancing. To mitigate residual imbalance during recovery, LUMEN identifies overloaded checkpoint holders using an *average-based request-count rule*: it compares each surviving worker’s total request load (i.e., queued, running, and newly assigned interrupted requests) against the cluster-wide average computed after the initial recovery dispatch. When a holder is overloaded, LUMEN migrates its assigned requests to the least-loaded worker, prioritizing requests in increasing order of their actual checkpointed size (i.e., the volume of KV cache successfully persisted before the failure). This greedy heuristic minimizes recomputation cost: since migrating a request forfeits its local checkpoint and forces a full re-prefill, shedding requests with the smallest saved prefixes first limits the total lost state. The controller iteratively recomputes worker loads after each migration and always targets the most congested worker first, until no worker exceeds the cluster-wide average.

KV-reuse recovery. Following dispatch, checkpoint holders execute in-place recovery by exploiting preserved KV caches. From the request’s token history, the holder deterministically regenerates page tags to locate the longest contiguous checkpointed prefix within its local store, subsequently loading the

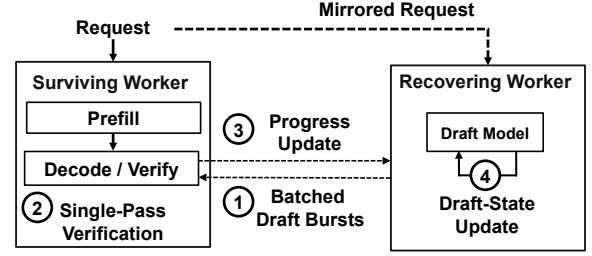


Figure 5: Speculation-assisted progressive recovery.

matching KV pages into GPU memory. If the persisted prefix is incomplete relative to the current token history, LUMEN bypasses full recomputation by performing a partial prefill on the uncheckpointed suffix. The restored KV pages serve as the context for this suffix prefill, enabling the request to seamlessly resume autoregressive decoding.

4.4 Speculation-assisted Progressive Recovery

LUMEN addresses Challenge 3 via speculation-assisted progressive recovery. The recovering worker moves through four internal states (Figure 5): **LOADING_DRAFT** loads the draft model; **ASSIST** feeds draft tokens to one paired surviving worker while the target model loads in the background; **HOTSWAP** transfers the preloaded target-model weights to the GPU; and **FULL_SERVICE** resumes normal target-model serving. This ensures draft assistance and target-model loading overlap, and the worker returns to full service without another disk-loading stall.

Draft-worker pairing. Upon entering the **ASSIST** state, LUMEN pairs the recovering worker exclusively with the surviving worker exhibiting the highest average queueing delay (§4.2), ensuring draft assistance is directed to the most congested worker. One-to-one pairing consolidates all draft tokens into a single batch, generating sufficient volume to amortize the target model’s verification overhead; unpaired surviving workers continue to receive new requests from the gateway transparently. To initiate assistance, the paired surviving worker transmits a *mirror request*, an exact token copy that seeds the draft model rather than producing user-facing output, for each in-flight request in its batch. The recovering worker continuously evaluates these mirror requests, accumulating unverified draft tokens until a target speculative depth of K is reached, at which point the tokens are transmitted back as a single burst (① in Figure 5). To minimize per-token coordination overhead, bursts completed within the same iteration are aggregated into a single network transfer.

Single-pass verification. A naive implementation requires two target-model forward passes per decode step: one to verify drafted requests and another to decode unassisted ones. LUMEN circumvents this overhead by constructing a single fused batch per step (② in Figure 5). To maintain structural consistency, LUMEN enforces a uniform shape of $K + 1$ se-

sequence positions for every request: the first position holds the latest committed token, and the remaining K positions contain either draft tokens (for assisted requests) or dummy *placeholders* (for unassisted ones). This uniformity enables the entire batch to execute as a single forward pass within a single CUDA graph. For unassisted requests, only the first position produces the next token, while placeholder outputs are discarded, mimicking a standard decode step. For assisted requests, the worker applies standard speculative decoding [21], accepting the longest valid prefix and appending the corrected token at the first rejection point. As one-to-one pairing concentrates draft tokens into this single batch, the vast majority of requests are assisted, rendering the computational overhead of placeholders negligible.

Draft-state alignment. To maintain asynchronous execution, the paired surviving worker never blocks the recovering worker. Instead, after each decode step, it transmits a progress update (③ in Figure 5) that includes the newly committed tokens and the output lengths for all mirror requests. Upon receiving this update, the recovering worker aligns its draft KV cache by sequence position (④ in Figure 5). Specifically, it compares the authoritative committed tokens against its local draft tokens; the draft KV remains valid only up to the first mismatched position. From that divergence point onward, the recovering worker truncates the invalid draft KV, replays the remaining committed tokens through the draft model to rebuild the context, and resumes speculative generation. Strict positional alignment is essential, as value-based matching is ambiguous due to token recurrence across different sequence positions.

Background loading and final switch. To circumvent prolonged synchronous stalls, LUMEN initiates a background process to load target-model weights into host memory concurrently with the ASSIST state. This masks the high-latency disk-to-host I/O and isolates only the rapid host-to-GPU memory transfer onto the HOTSWAP critical path. Upon completion of host-side loading, the recovering worker discards its transient draft state, deallocates the draft runtime, and transfers the preloaded target weights to the GPU to resume FULL_SERVICE. This transition is seamless: the authoritative request state and token history remain anchored at the surviving worker and gateway, respectively, so no state needs to be migrated at switch time. The pipeline also degrades gracefully under adverse conditions: cache misses fall back to re-prefill, lagging draft bursts are ignored by the surviving worker without stalling decode, and unexpected background loading delays simply extend the ASSIST phase until the hot-swap can proceed safely.

4.5 Discussion

Trade-offs. LUMEN incurs negligible steady-state overhead, primarily consuming host memory and asynchronous net-

work bandwidth for KV checkpointing (§6). Locality-aware scheduling may induce recomputation penalties by redirecting requests away from overloaded checkpoint holders; however, this cost is minimized by migrating requests with the shortest checkpointed prefixes. Speculation-assisted recovery introduces a transient draft-model footprint and bounded verification overhead, constrained by the one-to-one worker pairing. Finally, LUMEN currently maintains a single KV checkpoint per request, falling back to full recomputation if the checkpoint holder itself fails; we defer multi-checkpoint replication to future work.

Multi-worker failures. LUMEN consolidates concurrent multi-worker failures into a unified recovery event. The gateway aggregates all interrupted requests and executes locality-aware recovery scheduling across the surviving cluster. Concurrently, each recovering worker independently initiates speculation-assisted recovery, prioritizing pairings with surviving workers exhibiting the highest queueing delays. To prevent verification bottlenecks, the controller enforces strict one-to-one assignments: if multiple recovering workers target the same node, it reroutes the latter to the next most congested, unpaired worker. If all surviving workers are actively paired, any remaining recovering workers bypass draft assistance and directly load the full target model.

Gateway and controller failures. LUMEN assumes the gateway and controller remain operational during recovery. Although logically centralized, both components can achieve high availability and fault tolerance via standard replication.

5 Implementation

Prototype implementation. We implement the LUMEN prototype atop SGLang (v0.5.6) [43] in Python (~13 K LoC), leveraging PyTorch (v2.9) [7], FlashInfer (v0.5) [2], ZeroMQ [8], and gRPC (v1.75) [3]. We integrate four core components into SGLang: (i) a centralized controller for checkpoint holder assignment; (ii) a ZMQ-based transport layer for streaming newly generated KV pages to remote host memory; (iii) a recovery-aware Sarathi-Serve scheduler [10] that partitions traffic into KV-reuse, recomputation, and new-request queues; and (iv) a progressive recovery pipeline that executes the draft model, synchronizes tokens via gRPC, and hot-swaps the target model upon background loading completion. Additionally, we augment the SGLang gateway with load-aware routing logic, parsing health checks to track worker states and isolating non-FULL_SERVICE workers from new traffic, while explicitly permitting mirror requests for workers in the ASSIST state.

Simulator implementation. For large-scale evaluation, we extend Vidur [9] (~5 K LoC) with five modules: (i) an event-driven worker state machine governing recovery transitions (§4.4); (ii) a global scheduler for post-failure request dispatch; (iii) a recovery-aware Sarathi-Serve scheduler [10] mirroring

the prototype’s queue abstraction; (iv) a KV-restore analytical model that replaces Vidur’s default prefill-recomputation penalty with bandwidth-derived KV-load times; and (v) a KV-distribution module to execute load-aware checkpoint placement decisions across the cluster.

6 Evaluation

We evaluate LUMEN using both a hardware prototype and a large-scale simulator: the prototype measures real system behavior (e.g., disk I/O, network transfer, and GPU memory management under failures), while the simulator enables controlled configurations across cluster sizes, failure counts, and load regimes beyond the reach of our physical testbed.

6.1 Methodology

Evaluation environments. The prototype runs on a cluster of four nodes. Each node has two Intel Xeon Platinum 8358P processors (32 cores each at 2.6 GHz; 10 cores per socket allocated to our experiments), 200 GB DRAM, two 894 GB Samsung PM893 SATA SSDs, and two NVIDIA A800-SXM4-80 GB GPUs connected by NVLink at 200 GB/s inter-GPU bandwidth per GPU and attached to the host over PCIe 4.0 $\times$ 16; the nodes are interconnected by 10 Gbps Ethernet. The cluster provides eight A800 GPUs in total. We evaluate two deployments: (i) four workers (each spanning the two GPUs of one node under tensor parallelism) serving Qwen3-32B with a Qwen3-4B draft model [41], and (ii) eight workers (one GPU per worker, two workers per node) serving Qwen3-14B with a Qwen3-1.7B draft model [41].

The simulator runs on a server with two Intel Xeon Gold 6330 processors (28 cores each at 2.0 GHz) and 503 GB DRAM. By default, the simulator models a deployment with ten workers (each spanning four A100-80 GB-class GPUs) serving Llama-3-70B with a Llama-3-8B draft model [24] at a measured acceptance rate of 0.60. It uses 26 GB/s GPU-to-CPU bandwidth [14] and 2 GB/s local-SSD bandwidth [31].

Workloads. The prototype uses the ShareGPT trace [30], and the simulator uses the Splitwise-Conv trace [26]; the two datasets exercise complementary operating regimes (longer, conversational sessions in ShareGPT versus shorter, high-concurrency prefill-decode phases in Splitwise-Conv).

Default settings. Unless otherwise stated, both environments use a speculative depth of $K = 4$, the Sarathi-Serve scheduler [10] with continuous batching [42], a chunk size of 1,024 tokens, and a batch cap of 512. Each setup issues 15,000 requests (40,000 for Experiments B.4 and B.5 over a longer trace) under Poisson arrival. The default request rates are 12 QPS (four-worker), 10 QPS (eight-worker), and 14 QPS (simulator). We inject a single worker failure per deployment.

Baselines. We compare LUMEN against three schemes: No-Failure (running the same workload without failure injection),

Table 2: Experiment A.1 (Prototype): Recovery time, with latency breakdown for interrupted (Int) and uninterrupted (Unint) requests under the eight-worker Qwen3-14B deployment.

Scheme	Recovery (s)	Mean TTFT (s)		Mean TPOT (ms)	
		Int	Unint	Int	Unint
No-Failure	—	—	0.83 $\pm$ 0.23	—	92.6 $\pm$ 2.1
S&R	83.3	12.5 $\pm$ 3.6	1.3 $\pm$ 0.48	116.5 $\pm$ 6.6	99.3 $\pm$ 3.1
F-Ckpt	82.8	10.7 $\pm$ 3.1	1.1 $\pm$ 0.37	59.7 $\pm$ 9.6	97.2 $\pm$ 2.6
LUMEN	29.9	11.0 $\pm$ 3.4	0.87 $\pm$ 0.26	58.0 $\pm$ 12.4	93.1 $\pm$ 2.3

Stop-and-Restart (S&R), and Fixed-Checkpointing (F-Ckpt) (both defined in §1).

Metrics. We bucketize a sequence of request IDs (200 requests per bucket) and measure the mean/P99 TTFT and TPOT per bucket. We identify the *failure-impact window* for each recovery scheme, starting when the bucketized mean TTFT exceeds that of the aligned No-Failure bucket by more than 5%, and continuing until three consecutive buckets fall back within 5%. We define the recovery time as the duration of the failure-impact window. We present average results over five runs, with 95% confidence intervals under the Student’s t-distribution shown in tables and bar charts; confidence intervals are omitted from line plots for brevity.

6.2 Prototype Evaluation

Experiment A.1 (End-to-end recovery). Figure 6 shows bucketized mean and P99 TTFT and TPOT for all schemes under a single-worker failure. All three recovery schemes increase latency above No-Failure after the failure, but LUMEN recovers fastest: it has the lowest peak latency and returns to the baseline first. Stop-and-Restart has the highest peak (mean TTFT above 5 s at four workers, dropping below 1 s before the failure), as it stops serving and recomputes the lost prefill from scratch, whereas Fixed-Checkpointing and LUMEN keep the peak much lower by reusing the checkpointed KV cache. The failure increases latency more at four workers than at eight workers because it removes a larger share of serving capacity (25% vs. 12.5%). The P99 metrics show the same trend with more noise; the comparisons below use failure-impact window means.

Table 2 decomposes the failure-impact window by interrupted and uninterrupted requests; we focus on the eight-worker deployment, while the results for the four-worker case are consistent. Interrupted requests are few (1.7%); for example, their mean TTFT under Stop-and-Restart is 9.6 $\times$ that of uninterrupted requests within the failure-impact window because the failed prefill is discarded and recomputed from scratch. As TTFT is measured from each request’s original arrival time, the mean TTFT for interrupted requests varies significantly and is similar under LUMEN and Fixed-Checkpointing; LUMEN’s advantage for this population is in TPOT rather than TTFT. LUMEN reduces their mean TPOT

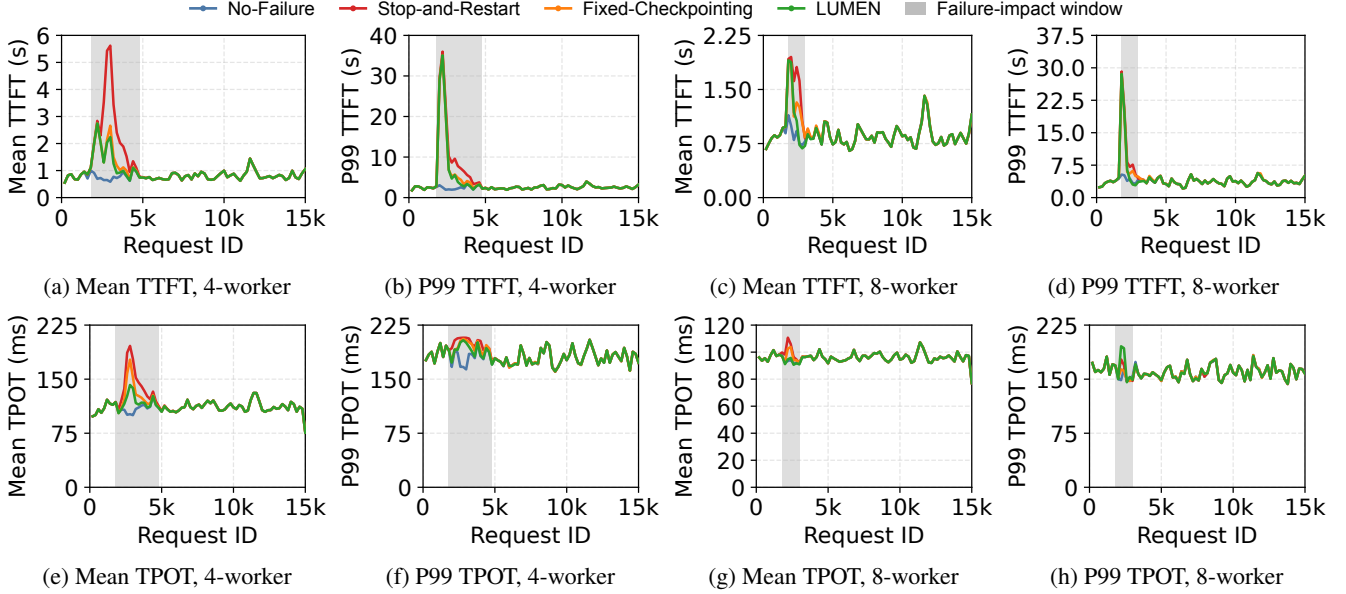


Figure 6: Experiment A.1: (Prototype) End-to-end recovery under a single-worker failure for four-worker (Qwen3-32B) and eight-worker (Qwen3-14B) deployments. We show the failure-impact window (in grey box) of Stop-and-Restart. Note that outside the failure-impact window (i.e., no failures), each request runs the same query workflow, so the curves overlap.

by 50.2% over Stop-and-Restart: resuming decode on restored KV cache already brings Fixed-Checkpointing to 59.7 ms, and the speculation path reduces it further to 58.0 ms. For the dominant uninterrupted requests (98.3%), LUMEN reduces mean TTFT by 33.1% over Stop-and-Restart, confirming that fast recovery benefits a much larger set of requests that never run on the failed worker.

LUMEN adds no measurable steady-state overhead before the failure: its curves overlap No-Failure on both TTFT and TPOT, with mean TTFT of 0.76 s on the four-worker deployment and 0.79 s on the eight-worker deployment (identical to No-Failure in both cases), and mean TPOT matching within 0.1 ms. Since this end-to-end measurement already includes the controller’s checkpoint placement and load-report handling, it confirms that the controller adds no measurable overhead at prototype scale.

Over the failure-impact window, LUMEN achieves the largest latency reduction. It reduces mean TTFT by 44.4% on the four-worker deployment and by 29.6% on the eight-worker deployment, outperforming Fixed-Checkpointing in both cases. Mean TPOT drops by 15.9% (four workers) and 7.1% (eight workers) over Stop-and-Restart, because the speculation path handles decode while the recovering worker’s target model is still loading. Furthermore, LUMEN reduces recovery time by 50.0% over Stop-and-Restart and by 34.9% over Fixed-Checkpointing on the four-worker deployment, and by 64.1% over Stop-and-Restart and by 63.9% over Fixed-Checkpointing on the eight-worker deployment. Fixed-Checkpointing recovers only slightly faster than Stop-and-Restart despite reusing its checkpointed KV cache, as it sends all of a failed worker’s restores to a single checkpoint holder

that then absorbs both its own traffic and the entire restore load; LUMEN avoids this concentration by spreading restore load across all surviving workers.

Experiment A.2 (Recovery-path breakdown). We decompose LUMEN’s recovery path to examine the contributions of individual techniques: **+Scheduling** enables the KV-reuse path, including load-aware KV checkpointing (§4.2) and locality-aware recovery scheduling (§4.3), so interrupted requests resume from their KV checkpoints; **+Progressive** enables only the speculative path (§4.4) without KV reuse; and **LUMEN** is the full system. Figure 7 compares various schemes over the failure-impact window on the eight-worker deployment. +Scheduling alone reduces mean TTFT by 19.1% by avoiding lost-prefix replay, whereas +Progressive alone reduces it by 18.5%; both fall below Stop-and-Restart. For mean TPOT, the two approaches contribute complementarily: +Scheduling reduces mean TPOT to 96.7 ms by resuming decode from restored KV pages, and +Progressive reduces it to 95.8 ms by handling decode through the speculation path. LUMEN combines both and achieves the lowest mean TTFT (1.0 s) and mean TPOT (92.5 ms).

Experiment A.3 (Impact of request rate). Figure 8 shows mean TTFT and mean TPOT across different request rates on the eight-worker deployment. Stop-and-Restart’s mean TTFT increases from 1.5 s at the default 10 QPS to 26.7 s at 12 QPS; at 8 QPS and 9 QPS, a single failure has limited impact, and the schemes differ by less than 0.2 s. LUMEN reduces mean TTFT by 29.6% over Stop-and-Restart at the default 10 QPS, with the reduction peaking at 33.3% at 11 QPS before saturation. Mean TPOT follows the same trend: LUMEN

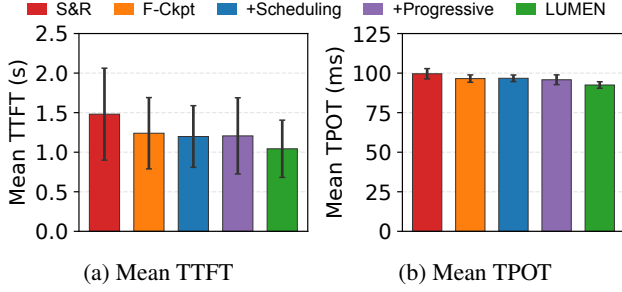


Figure 7: Experiment A.2 (Prototype): Recovery-path breakdown.

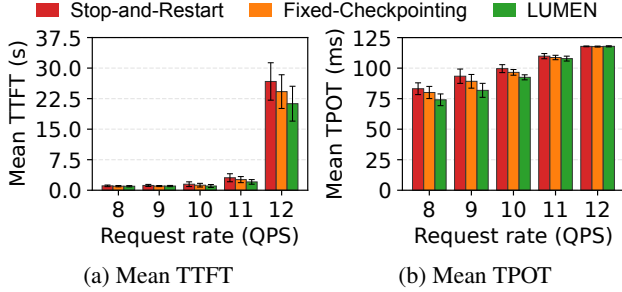


Figure 8: Experiment A.3 (Prototype): Impact of request rate.

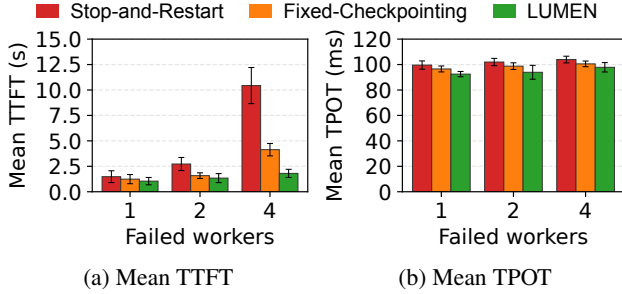


Figure 9: Experiment A.4 (Prototype): Impact of number of failures.

remains the lowest through the speculation path as queues grow, reducing mean TPOT by 10.9% at 8 QPS and by 7.1% at the default, while Stop-and-Restart’s TPOT increases sharply.

Experiment A.4 (Impact of number of failures). Figure 9 evaluates the impact of concurrent failures by failing one, two, and four out of eight workers. LUMEN’s gain grows with failure count, reducing mean TTFT over Stop-and-Restart by 29.6%, 50.8%, and 82.7% for one, two, and four failed workers, respectively. Stop-and-Restart’s mean TTFT increases sharply (1.5 s at one failure to 10.4 s at four) as its TPOT approaches the per-batch decode limit; LUMEN scales more gracefully as speculation and KV reuse together eliminate the prefill replay that dominates under high failure counts.

6.3 Simulator Evaluation

Experiment B.1 (End-to-end recovery). Table 3 reports end-to-end recovery on the 10-worker Llama-3-70B deployment under a single failure. LUMEN reduces mean TPOT within the failure-impact window by 22.6% over Stop-and-

Table 3: Experiment B.1 (Simulator): End-to-end recovery.

Scheme	Mean TTFT (s)	Mean TPOT (ms)	Recovery (s)
S&R	1.6 ± 0.34	167.1 ± 14.4	100.3
F-Ckpt	1.5 ± 0.26	156.8 ± 10.6	94.9
+Scheduling	1.5 ± 0.25	156.7 ± 9.7	91.1
+Progressive	1.6 ± 0.29	136.4 ± 9.9	81.5
LUMEN	1.4 ± 0.20	129.3 ± 6.0	81.5

Restart and 17.6% over Fixed-Checkpointing. Mean TTFT improvements are smaller (12.2% and 5.1% over the respective baselines) because a single failure across 10 workers raises TTFT only slightly while the cluster is unsaturated; this regime is consistent with the prototype at low load, and Experiment B.2 confirms larger TTFT gains as load increases. LUMEN also reduces recovery time by 18.7% over Stop-and-Restart and 14.1% over Fixed-Checkpointing. As recovery time is measured over the overall failure-impact window, the single-failure case is governed mostly by the many uninterrupted requests on surviving workers. Thus, +Progressive and LUMEN have almost identical recovery time: speculation restores decode capacity during target-model reload, while the extra KV-reuse path affects only the small interrupted-request set. KV reuse still appears in mean TPOT (5.2% below +Progressive), and its recovery-time benefit becomes visible when more workers fail (Experiment B.3).

The breakdown in Table 3 reveals complementary roles. +Scheduling reduces mean TTFT by 6.3% by eliminating lost-prefix replay, while +Progressive does not reduce mean TTFT in the rounded table (1.6 s, matching S&R) because the recovering worker cannot yet accept prefill while the draft model loads. For mean TPOT, roles are reversed: +Progressive contributes an 18.4% reduction by handling decode through speculation, while +Scheduling contributes only 6.2% because the recovering worker remains idle during target-model loading. LUMEN combines both features and achieves the lowest mean TTFT and mean TPOT.

Experiment B.2 (Impact of request rate). Figure 10 examines five request rates varied from 12 to 21 QPS, spanning near-saturation (12–16 QPS) and overload (17–21 QPS). In steady state, LUMEN’s gain is led by speculation: mean TPOT is reduced over Stop-and-Restart by 17.2–22.6% and over Fixed-Checkpointing by 14.2–17.6% across 12–15 QPS. Mean TTFT improves less at low load (5.3% at 12 QPS, 12.2% at 14 QPS over Stop-and-Restart), because a single failure barely perturbs an uncongested cluster. Under overload, the pattern reverses: growing queues amplify the TTFT gap (42.7% below Stop-and-Restart at 17 QPS), while the TPOT gap narrows as all schemes saturate.

Experiment B.3 (Impact of number of failures). Figure 11 varies one to five simultaneous failures across 10 workers. LUMEN’s advantage scales with failure count: at five failures, mean TTFT is 63.6% below Stop-and-Restart and 33.9% below Fixed-Checkpointing, mean TPOT is 31.5% and 23.0%

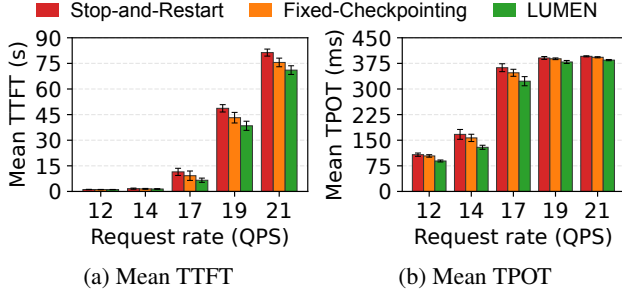


Figure 10: Experiment B.2 (Simulator): Impact of request rate.

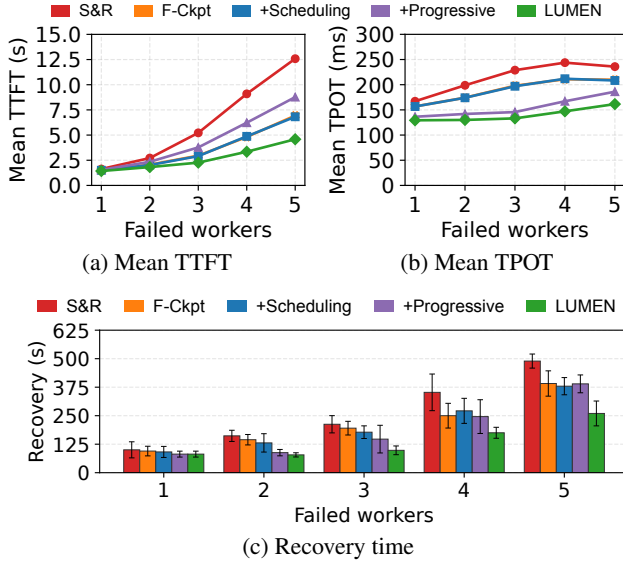


Figure 11: Experiment B.3 (Simulator): Impact of number of failed workers on TTFT, TPOT, and recovery time.

lower, and recovery time is 46.9% and 33.3% shorter, respectively. With a single failure, the schemes remain close as interrupted requests account for a small fraction of all requests in the failure-impact window; recovery time is governed mainly by uninterrupted requests queuing behind redirected traffic on surviving workers. As the failure count grows, a larger share of requests lose their serving worker, increasing the impact on cluster-wide recovery. +Scheduling stays close to Fixed-Checkpointing because the default request rate leaves short queues, preventing the fixed checkpoint holder from becoming a bottleneck; the benefit of load-aware placement is more pronounced under higher load, as confirmed in Experiment B.2.

Experiment B.4 (Scalability with proportional failures). Figure 12 scales the cluster from 4 to 64 workers with a fixed per-worker load of 1.4 QPS and a fixed 25% failure rate (i.e., 1–16 simultaneous failures). LUMEN’s gains are stable across all scales. Compared to Stop-and-Restart, mean TTFT stays 46.8–51.2% lower at 4–64 workers and mean TPOT is 30.7–43.8% lower over the same range; recovery time is shorter by 43.6–61.0%. Both KV reuse and progressive recovery

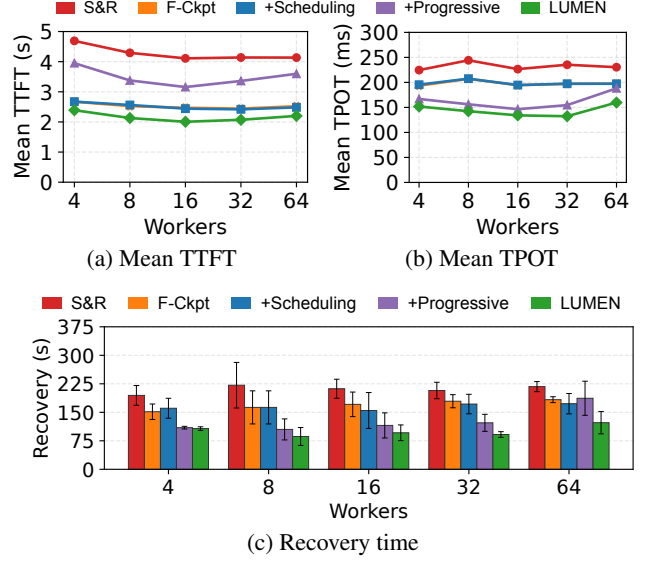


Figure 12: Experiment B.4 (Simulator): Scalability with proportional failures on TTFT, TPOT, and recovery time.

ery contribute: LUMEN outperforms Fixed-Checkpointing by 12.8–18.8% in mean TTFT and 19.2–32.9% in mean TPOT at 8–64 workers, with the difference attributable to the speculation path that Fixed-Checkpointing lacks. Note that under the fixed per-worker load, +Scheduling stays close to Fixed-Checkpointing because per-worker queueing pressure does not grow with cluster size, so the fixed checkpoint holder rarely becomes a bottleneck. LUMEN’s KV-reuse and progressive-recovery paths are therefore complementary: LUMEN reduces recovery time by 29.0–47.0% over +Scheduling (which lacks progressive recovery) and by 2.2–34.4% over +Progressive (which lacks KV reuse). Furthermore, under 16 failures out of 64 workers, 23.8% of interrupted requests fall back to full recomputation due to checkpoint-holder co-failure (§4.5).

Experiment B.5 (Scalability with a single failure). Figure 13 varies the cluster size while keeping a single failure, so the failure share decreases as the cluster size increases. LUMEN has the lowest latency at every scale, but its benefit shrinks as the single failure becomes a smaller fraction of the cluster. Stop-and-Restart’s mean TTFT within the window falls from 5.2 s at four workers to 1.2 s at 64, whereas under the fixed 25% failure fraction it remains within 4.1–5.2 s (Figure 12). As a result, LUMEN’s TTFT reduction over Stop-and-Restart falls from 50.6% at four workers to 1.3% at 64, and the TPOT reduction declines from 37.2% to 3.7% over the same range. Together, Experiments B.4 and B.5 span the full range of operating conditions: LUMEN removes much of the recovery cost when the failed fraction is high, and does not increase latency when a single failure is a negligible share of a large cluster. Table 4 further confirms that LUMEN reduces TPOT for interrupted requests by 53–67% over Stop-and-Restart across all scales through speculation.

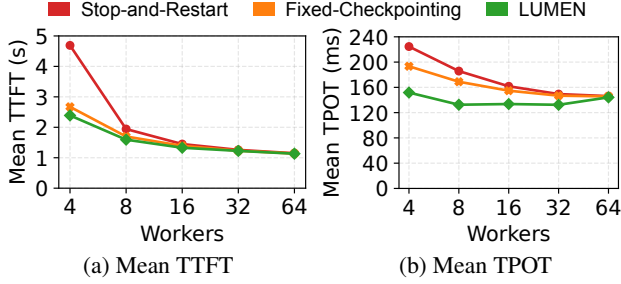


Figure 13: Experiment B.5 (Simulator): Scalability with a single failure.

Table 4: Experiment B.5: Per-request-type breakdown across worker counts.

Workers	Request type	Mean TTFT (s)			Mean TPOT (ms)		
		S&R	F-Ckpt	LUMEN	S&R	F-Ckpt	LUMEN
4	Interrupted	27.51	23.61	23.44	357.7	176.9	116.5
	Uninterrupted	4.10	2.14	1.85	231.6	200.9	154.6
8	Interrupted	24.26	22.26	22.08	236.1	115.7	79.6
	Uninterrupted	1.55	1.34	1.24	177.2	163.8	133.8
16	Interrupted	22.08	21.20	20.61	177.7	129.9	76.4
	Uninterrupted	1.25	1.20	1.14	157.9	151.8	135.6
32	Interrupted	26.28	25.55	25.36	150.0	87.8	60.0
	Uninterrupted	1.11	1.09	1.09	147.1	143.8	133.8
64	Interrupted	23.43	22.60	22.47	147.6	82.0	69.6
	Uninterrupted	1.05	1.05	1.04	141.7	140.5	136.9

Table 5: Experiment B.6 (Simulator): LUMEN’s TTFT and TPOT across speculative depth K , each paired with its acceptance rate α .

	$K = 2$	$K = 4$	$K = 8$
Acceptance rate α	0.72	0.60	0.50
Mean TTFT (s)	1.4 ± 0.20	1.4 ± 0.20	1.4 ± 0.20
Mean TPOT (ms)	130.1 ± 6.2	129.3 ± 6.0	128.9 ± 5.9

Table 6: Experiment B.7 (Simulator): LUMEN’s TTFT and TPOT across the checkpoint-placement weight λ .

λ	0.25	0.5	1	2	4
TTFT (s)	1.4 ± 0.20	1.4 ± 0.20	1.4 ± 0.20	1.4 ± 0.20	1.4 ± 0.21
TPOT (ms)	129.0 ± 5.9	129.0 ± 6.0	129.0 ± 5.7	129.1 ± 5.7	129.2 ± 5.9

Experiment B.6 (Speculative-depth sensitivity). Table 5 varies K over (2, 4, 8) draft tokens per step using the measured acceptance rates (0.72, 0.60, 0.50) for the Llama-3-70B pair. LUMEN remains insensitive to K : its mean TTFT stays at 1.4 s and mean TPOT changes by only 0.9% across K , both within confidence intervals. This insensitivity is inherent to speculative decoding: a deeper draft proposes more tokens per step, but the correspondingly lower acceptance rate cancels the gain, keeping the expected number of accepted tokens near constant (around 1.2, 1.3, and 1.0 under the respective acceptance rates) while each verification step becomes more expensive. The default $K = 4$ is robust.

Experiment B.7 (Checkpoint-placement weight sensitivity). Table 6 varies λ (Equation (1)) from 0.25 to 4. LUMEN’s mean TTFT stays at 1.4 s and mean TPOT remains within 129.0–129.2 ms across λ (under 0.5% variation, within confidence intervals). Under a single-worker failure, λ only affects

where the KV checkpoints of the few interrupted requests are placed; mean TTFT and TPOT within the failure-impact window are governed primarily by the surviving workers absorbing the failed worker’s traffic. The default $\lambda = 1$ is robust for single-failure conditions.

6.4 Highlights of Evaluation Results

LUMEN’s performance gains over Stop-and-Restart and Fixed-Checkpointing are most pronounced under three conditions: (i) high failure severity, where mean TTFT reductions over Stop-and-Restart grow from 29.6% at one failed worker to 82.7% at four on the prototype (Experiment A.4) and from 12.2% to 63.6% on the simulator (Experiment B.3); (ii) high cluster load, where near-saturation queues amplify the TTFT advantage to 42.7% over Stop-and-Restart (Experiment B.2); and (iii) proportionally large failure fractions, where gains remain stable at 46.8–51.2% below Stop-and-Restart in mean TTFT across 4 to 64 workers at a fixed 25% failure rate (Experiment B.4). In all conditions, LUMEN adds no measurable steady-state overhead.

7 Related Work

Fault tolerance in LLM training. Gemini [38] shortens training recovery time by storing in-memory checkpoints of model weights and optimizer states on other workers’ hosts. Bamboo [34], Ooblick [16], and Recycle [13] achieve resilient training through redundant pipeline execution and template-based reconfiguration. ECCheck [27] lowers the memory cost of in-memory checkpoints through erasure coding. ByteCheckpoint [37] unifies large foundation-model training checkpointing across parallelism strategies. However, the above training studies target long-running training jobs; they do not address per-request state under interactive latency requirements as LUMEN does.

Fault tolerance in LLM serving. Recent work makes LLM serving resilient to GPU failures by reusing previously computed KV cache. DéjàVu [32] streams KV cache to another worker’s CPU memory through a fixed ring topology and always assigns each worker’s checkpoint to a fixed neighbor. FailSafe [40] sustains intra-node serving under irregular GPU availability, while GhostServe [17] protects KV cache with erasure-coded parity shards in host memory. KevlarFlow [28] addresses multi-worker recovery through GPU-memory KV replication and pipeline reconfiguration, but does not route recovery in a load-aware manner. LUMEN instead distributes recovery work using recent worker-load observations and overlaps full-model loading with draft-generation assistance to keep the recovering worker productive.

Efficient LLM scheduling. Orca [42] proposes iteration-level scheduling for transformer serving. vLLM [20] introduces paged KV management, which LUMEN builds on for

request-state movement and reuse. Sarathi-Serve [10] improves inference efficiency with chunked prefills and decode piggybacking. SGLang [43] provides an efficient execution substrate for structured language model programs, and LUMEN builds directly on this runtime. Llumnix [33] supports live request migration for dynamic scheduling. FastServe [39] introduces preemptive scheduling with proactive job swapping to minimize latency. However, all the above studies assume worker availability in the steady state and do not enable a recovering worker to serve before its model is fully reloaded. LUMEN’s load-aware recovery is complementary to these schedulers without modifying their steady-state scheduling logic.

8 Conclusion

We study the failure recovery problem for distributed LLM serving, where worker failures lead to the loss of both in-flight KV cache and serving capacity, thereby degrading both TTFT and TPOT. LUMEN is a fault-tolerant LLM serving system that makes *load-aware* recovery decisions. It incorporates three novel techniques: load-aware KV checkpointing, locality-aware recovery scheduling, and speculation-assisted progressive recovery. Our prototype and simulation results show that LUMEN reduces TTFT, TPOT, and recovery time compared to the recovery baselines Stop-and-Restart and Fixed-Checkpointing across various settings.

References

- [1] Deploying vLLM on Kubernetes. <https://docs.vllm.ai/en/stable/deployment/k8s/>, 2026.
- [2] FlashInfer: Kernel library for LLM serving. <https://github.com/flashinfer-ai/flashinfer>, 2026.
- [3] gRPC: A high-performance, open-source universal RPC framework. <https://grpc.io>, 2026.
- [4] Hugging Face text generation inference. <https://huggingface.co/docs/text-generation-inference>, 2026.
- [5] KServe: Standardized model inference platform on Kubernetes. <https://github.com/kserve/kserve>, 2026.
- [6] NVIDIA Triton inference server. <https://github.com/triton-inference-server/server>, 2026.
- [7] PyTorch: An open-source machine learning framework. <https://pytorch.org>, 2026.
- [8] ZeroMQ: An open-source universal messaging library. <https://zeromq.org>, 2026.
- [9] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S Gulavani, Ramachandran Ramjee, and Alexey Tumanov. Vidur: A large-scale simulation framework for LLM inference. In *Conference on Machine Learning and Systems (MLSys)*, 2024.
- [10] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [11] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [12] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. <https://arxiv.org/abs/2302.01318>, 2023.
- [13] Swapnil Gandhi, Mark Zhao, Athinagoras Skiadopoulos, and Christos Kozyrakis. Recycle: Resilient training of large DNNs using pipeline adaptation. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2024.
- [14] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In *USENIX Annual Technical Conference (ATC)*, 2024.
- [15] Qinghao Hu, Zhisheng Ye, Zerui Wang, Guoteng Wang, Meng Zhang, Qiaoling Chen, Peng Sun, Dahua Lin, Xiaolin Wang, Yingwei Luo, Yonggang Wen, and Tianwei Liu. Characterization of large language model development in the datacenter. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024.
- [16] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. Oobleck: Resilient distributed training of large models using pipeline templates. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.

- [17] Shakya Jayakody, Youpeng Zhao, Chinmay Nehate, and Jun Wang. GhostServe: A lightweight checkpointing system in the shadow for fault-tolerant LLM serving. In *Conference on Machine Learning and Systems (MLSys)*, 2026.
- [18] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. MegaScale: Scaling large language model training to more than 10,000 GPUs. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024.
- [19] Apostolos Kokolis, Michael Kuchnik, John Hoffman, Adithya Kumar, Parth Malani, Faye Ma, Zachary DeVito, Shubho Sengupta, Kalyan Saladi, and Carole-Jean Wu. Revisiting reliability in large-scale machine learning research clusters. In *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2025.
- [20] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with Page-dAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [21] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.
- [22] Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. PEARL: Parallel speculative decoding with adaptive draft length. In *International Conference on Learning Representations (ICLR)*, 2025.
- [23] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Liu, Shan Lu He, Ganesh Ananthanarayanan, and Junchen Jiang. CacheGen: KV cache compression and streaming for fast large language model serving. In *ACM SIGCOMM Conference (SIGCOMM)*, 2024.
- [24] Llama Team. The Llama 3 herd of models. <https://arxiv.org/abs/2407.21783>, 2024.
- [25] Bradley McDanel. AMUSD: Asynchronous multi-device speculative decoding for LLM acceleration. In *IEEE International Symposium on Circuits and Systems (ISCAS)*, 2025.
- [26] Pratyush Patel, Esha Choukse, Chaojie Zhang, Inigo Goiri, Aashaka Shah, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative LLM inference using phase splitting. In *IEEE International Symposium on Computer Architecture (ISCA)*, 2024.
- [27] Guicheng Qi, Zongpeng Li, Chuan Wu, Zhuwei Peng, and Yi Zheng. ECCheck: Enhancing in-memory checkpoint with erasure coding in distributed DNN training. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2025.
- [28] Shangshu Qian, Kipling Liu, PC Sruthi, Lin Tan, and Yongle Zhang. Towards resiliency in large language model serving with KevlarFlow. <https://arxiv.org/abs/2601.22438>, 2026.
- [29] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Wang, Jiaao Zeng, Yongchao Zhang, Jian Shi, Chengruidong Bian, Lian Yang, Fan Lv, Yang Luo, and Zhongzhi Yang. Mooncake: Trading more storage for less computation – a KVCache-centric architecture for serving LLM chatbot. In *USENIX Conference on File and Storage Technologies (FAST)*, 2025.
- [30] ShareGPT. ShareGPT conversation dataset. <https://sharegpt.com/>.
- [31] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning (ICML)*, 2023.
- [32] Foteini Strati, Sara McAllister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. DéjàVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. In *International Conference on Machine Learning (ICML)*, 2024.
- [33] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [34] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. Bamboo: Making preemptible instances resilient for affordable training of large DNNs. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023.
- [35] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale,

- Dan Bikel, Lukas Blecher, Cristian Canton-Ferrer, Moya Chen, Guillem Cucurull, David Esiobu, Jude Fernandes, Jeremy Fu, Wenyin Fu, Brian Fuller, Cynthia Gao, Vedanuj Goswami, Naman Goyal, Anthony Hartshorn, Saghar Hosseini, Rui Hou, Hakan Inan, Marcin Kardas, Viktor Kerkez, Madian Khabsa, Isabel Kloumann, Artem Korenev, Punit Singh Koura, Marie-Anne Lachaux, Thibaut Lavril, Jenya Lee, Diana Liskovich, Yinghai Lu, Yuning Mao, Xavier Martinet, Todor Mihaylov, Pushkar Mishra, Igor Molybog, Yixin Nie, Andrew Poulton, Jeremy Reizenstein, Rashi Rungta, Kalyan Saladi, Alan Schelten, Ruan Silva, Eric Michael Smith, Rannan Subramanian, Xiaoqing Ellen Tan, Binh Tang, Ross Taylor, Adina Williams, Jian Xiang Kuan, Puxin Xu, Zheng Yan, Iliyan Zarov, Yuchen Zhang, Angela Fan, Melanie Kambadur, Sharan Narang, Aurélien Rodriguez, Robert Stojnic, Sergey Edunov, and Thomas Scialom. Llama 2: Open foundation and fine-tuned chat models. <https://arxiv.org/abs/2307.09288>, 2023.
- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [37] Borui Wan, Mingji Han, Yiyao Sheng, Yanghua Peng, Haibin Lin, Mofan Zhang, Zhichao Lai, Menghan Yu, Junda Zhang, Zuquan Song, Xin Liu, and Chuan Wu. ByteCheckpoint: A unified checkpointing system for large foundation model development. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2025.
- [38] Zhuang Wang, Zhen Jia, Shuai Zheng, Zhen Zhang, Xinwei Fu, T. S. Eugene Ng, and Yida Wang. Gemini: Fast failure recovery in distributed training with in-memory checkpoints. In *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [39] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. Fast distributed inference serving for large language models. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2026.
- [40] Ziyi Xu, Zhiqiang Xie, Swapnil Gandhi, and Christos Kozyrakis. FailSafe: High-performance resilient serving. In *Conference on Machine Learning and Systems (MLSys)*, 2026.
- [41] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jinguang Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report. <https://arxiv.org/abs/2505.09388>, 2025.
- [42] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for transformer-based generative models. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- [43] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [44] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [45] Yazhou Zu, Alireza Ghaffarkhah, Hoang-Vu Dang, Brian Towles, Steven Hand, Safeen Huda, Adekunle Bello, Alexander Kolbasov, Arash Rezaei, Dayou Du, Steve Lacy, Hang Wang, Aaron Wisner, Chris Lewis, and Henri Bahini. Resiliency at scale: Managing Google’s TPUv4 machine learning supercomputer. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2024.